

SYSTEM ANALYST

presented by Ayub

Requirements Engineering A

Кейс из жизни

Ситуация:

- Заказчик: "Сделайте мне приложение для доставки еды"
- Команда: "Ок!" (работают 3 месяца)
- Релиз → Показываем заказчику
- Заказчик: "Это вообще не то, что я хотел!"

Знакомо?

Почему так произошло? Что пошло не так в коммуникации между заказчиком и командой?

Корень проблемы

Главная проблема:

- Не было **чёткого описания** того, ЧТО именно нужно делать
- Каждый представлял что-то своё в голове
- Никто не записал и не согласовал детали

Решение:

- **Требования** — это и есть такое описание
- Записанное, согласованное, понятное всем

Требования — это мост между идеей заказчика и работой команды

Представь, что ты заказываешь пиццу по телефону. Что ты должен сказать курьеру, чтобы получить ИМЕННО то, что хочешь?

Определение

Требование — это описание того, что система должна делать (или каким свойством обладать).

Ключевые свойства хорошего требования:

1. **Измеримое** — можно проверить цифрой или тестом
2. **Проверяемое** — понятно как протестировать выполнено ли
3. **Однозначное** — все понимают одинаково
4. **Необходимое** — есть причина зачем это нужно
5. **Реалистичное** — технически можно сделать

Хорошее vs Плохое требование

Плохое требование	Почему плохо	Хорошее требование
Система должна быть быстрой	Что значит "быстрая"?	Страница загружается за < 2 сек
Интерфейс должен быть удобным	Как проверить?	90% находят кнопку за < 10 сек
Должно быть много товаров	Сколько это "много"?	Каталог содержит мин. 1000 SKU
Без ошибок	Слишком общо	Критичные падения < 1 раза/мес

Упражнение

Оцени требования — хорошие или плохие?

1. "Пользователь должен иметь возможность сбросить пароль"
2. "Форма регистрации должна быть красивой"
3. "SMS с кодом приходит в течение 30 секунд после запроса"
4. "Система должна поддерживать много пользователей одновременно"

Module Check

1. Что такое требование одним предложением?
2. Назови 3 свойства хорошего требования
3. Почему требование "система должна быть надёжной" — плохое?
4. Как превратить "сайт должен грузиться быстро" в хорошее требование?

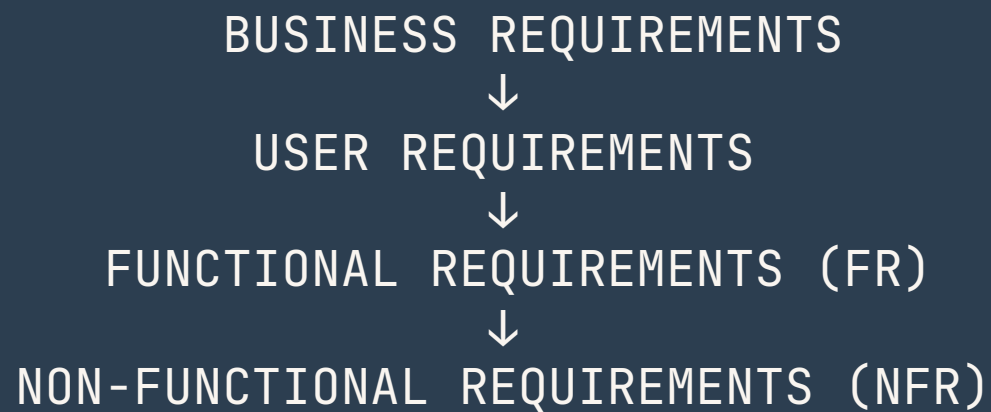
Когда открывается новая кофейня, кто решает ЧТО там будет?

- А) Бариста (тот кто варит кофе)
- В) Владелец кофейни
- С) Посетители

И ЧТО именно каждый из них хочет?

Пирамида требований

4 уровня требований (сверху вниз):



Связь: каждый уровень вытекает из предыдущего

Business Requirements

Business Requirements — зачем бизнесу этот проект?

Характеристики:

- Формулируются владельцами бизнеса
- Выражены в бизнес-терминах (деньги, время, доля рынка)
- Измеримы цифрами KPI
- Не про технологию, про деньги/клиентов/рынок

Примеры:

- "Увеличить онлайн-продажи на 25% за 6 месяцев"
- "Сократить время обработки заказа с 10 минут до 2 минут"
- "Привлечь 50 000 новых пользователей в первый квартал"

User Requirements

User Requirements — что пользователь хочет ДЕЛАТЬ?

Характеристики:

- Описывают действия/задачи пользователя
- Пишутся языком пользователя (не технический жаргон)
- Фокус на ЗАЧЕМ пользователь это делает (ценность)
- Часто в формате User Story

Примеры:

- "Зарегистрироваться в системе через email"
- "Найти товар по категории и цене"
- "Отследить статус доставки заказа"
- "Оставить отзыв о товаре"

Functional Requirements (FR)

Functional Requirements — конкретное ПОВЕДЕНИЕ системы

Характеристики:

- Описывают ЧТО система делает (действие, функция)
- Формат: "Когда пользователь делает X, система делает Y"
- Проверяемы тестами
- Детализируют User Requirements

Примеры:

- "При нажатии 'Купить' система создаёт заказ и отправляет email"
- "Система блокирует аккаунт после 5 неудачных попыток входа"
- "При вводе промокода система пересчитывает сумму со скидкой"

Non-Functional Requirements (NFR)

Non-Functional Requirements — КАК система работает (качество)

Категории NFR:

- **Performance** — скорость, время отклика
- **Security** — защита данных, аутентификация
- **Usability** — удобство использования
- **Reliability** — стабильность, uptime
- **Scalability** — способность расти под нагрузкой

Примеры:

- "API отвечает за < 200 мс в 95% запросов"
- "Пароли хранятся в зашифрованном виде (bcrypt)"
- "Сайт доступен 99.9% времени"

FR vs NFR — В чём разница?

Критерий	FR	NFR
Вопрос	ЧТО делает?	КАК делает?
Фокус	Функции, действия	Качество, свойства
Пример	Можно сбросить пароль	Сброс за < 5 секунд
Тестирование	Функциональные тесты	Нагрузочные, security
Если нарушено	Не работает	Работает, но плохо

Пирамида в действии — Пример

Кейс: Интернет-магазин одежды

1. **Business Req**: Увеличить конверсию с 2% до 5% за квартал
2. **User Req**: Видеть товар на модели моего размера
3. **FR**: При выборе размера фото обновляется на соответствующий
4. **NFR**: Фото загружается за < 1 сек, работает на мобильных

Видишь связь? Всё идёт от бизнес-цели вниз к деталям реализации

Module Check

1. Назови 4 уровня пирамиды требований сверху вниз
2. В чём разница между Business и User Requirements?
3. Это FR или NFR: "Система отправляет SMS при создании заказа"?
4. Это FR или NFR: "SMS доставляется за < 30 секунд"?
5. Придумай по 1 примеру FR и NFR для банкомата

Final Check: Урок А

1. Зачем вообще нужны требования? Что будет если их не писать?
2. Назови 3 свойства хорошего требования и объясни каждое
3. Приведи пример плохого требования и переделай его в хорошее
4. Объясни разницу FR и NFR своими словами + по 1 примеру
5. Как Business Requirements связаны с User Requirements?

Ресар: Урок А

Концепция	Суть	Пример
Требование	Описание что/как система должна работать	"Страница грузится < 2 сек"
Свойства	Измеримое, проверяемое, однозначное	Цифры, тесты
Business Req	Зачем бизнесу проект	"Увеличить продажи на 20%"
User Req	Что пользователь хочет делать	"Найти товар по фильтру"
FR	Конкретное поведение системы	"Кнопка создаёт заказ"
NFR	Качество работы системы	"API отвечает < 200 мс"

Требования — это фундамент проекта. Плохие требования → плохой продукт.

Вопросы?